

ANDROID DEVELOPER INTERNSHIP PROGRAM

Task 6

Task Title:

Plan API Structure & Data Flow (REST Integration Strategy)

App Name: Eco-Cart

Proposed by: Ruhma Amir

1. Identify API Requirements

The Eco-Cart app requires communication with a backend for the following core functions:

- **User Authentication:** Handling sign-up, login, and secure session management.
- **Product Lookup:** Fetching detailed sustainability data, carbon footprint values, and eco-scores using a barcode.
- **Activity Logging:** Sending scan events to the server to maintain user history.
- **Eco-Insights:** Retrieving personalized statistics and "green" recommendations.

2. Define REST API Endpoints

We will use standard HTTP methods to interact with the resources.

Method	Endpoint	Purpose
POST	/auth/login	Authenticates user and returns a token.
GET	/products/{barcode}	Retrieves eco-details for a specific scanned item.
POST	/scans	Logs a new scan event to the user's history.
GET	/users/{uid}/impact	Fetches total CO2 saved and sustainability rank.
GET	/products/alternatives	Fetches "green" alternatives based on a category.

3. Request & Response Structure

Here is how the data packets look when moving between your app and the server.

Example: Fetching Product Details

Request (GET): <https://api.ecocart.com/v1/products/890123456789>

Response (JSON):

JSON

```
{
  "barcode": "890123456789",
  "name": "Organic Almond Milk",
  "eco_score": "A",
  "carbon_val": 0.52,
  "recommendation": "High sustainability rating. Low water usage."
}
```

4. Data Flow Design

This diagram illustrates the "Journey of Data" from a user's camera scan to the screen update.

The Step-by-Step Flow:

1. **User Action:** User scans a barcode in the ScannerActivity (UI).
2. **ViewModel:** The ScannerActivity notifies the ProductViewModel.
3. **Repository:** The ViewModel calls the ProductRepository to get data.
4. **API Service:** The Repository uses **Retrofit** to send a GET request to the server.
5. **Server Response:** The server returns a JSON object.
6. **Data Processing:** The Repository parses the JSON into a Kotlin Data Class.
7. **UI Update:** The ViewModel updates a StateFlow or LiveData, which the UI observes to show the Product Details.

5. Error Handling Strategy

A professional app must handle failures gracefully:

1. **Network Failure:** If the internet is off, the app shows a "No Connection" snackbar and attempts to load cached data from the local database.
2. **API 404 (Not Found):** If a barcode isn't in the database, the app prompts the user to "Manually Enter Product Details."
3. **Token Expiry:** If the authentication token expires, the app automatically redirects the user to the Login screen.

6. Tools & Libraries Planning

To implement this strategy, the following stack will be used:

1. **Retrofit:** For defining and executing network requests.

2. **Moshi / Gson:** For converting JSON strings into Kotlin objects.
3. **OkHttp:** To handle logging (logging interceptors) and network timeouts.
4. **Kotlin Coroutines:** To perform network requests on a background thread without freezing the UI.